PRACT 4 Diabetes

November 2, 2026

[1]:
```
# Step 1: Import required libraries
import pandas as pd
import matplotlib.pyplot as plt
import seaborn as sns

from sklearn.model_selection import train_test_split
from sklearn.preprocessing import StandardScaler
from sklearn.neighbors import KNeighborsClassifier
from sklearn.metrics import confusion_matrix
from sklearn.metrics import accuracy_score
from sklearn.metrics import precision_score
from sklearn.metrics import ConfusionMatrixDisplay
```

[3]:
```
# Step 2: Load the dataset
df = pd.read_csv(r"E:\LP3_LAB Manual\ML PRAC\PRACT4\diabetes (1) - diabetes (1).
 ↪csv")

# Step 3: Display the dataset
print("First five rows:")
print(df.head())
```

```
First five rows:
   Pregnancies  Glucose  BloodPressure  SkinThickness  Insulin   BMI  \
0            6      148             72             35        0  33.6
1            1       85             66             29        0  26.6
2            8      183             64              0        0  23.3
3            1       89             66             23       94  28.1
4            0      137             40             35      168  43.1

   Pedigree  Age  Outcome
0     0.627   50        1
1     0.351   31        0
2     0.672   32        1
3     0.167   21        0
4     2.288   33        1
```

[4]:
```
# Step 4: Separate input features and target
X = df.drop("Outcome", axis=1)
```

```
y = df["Outcome"]

# Step 5: Split the dataset into training and testing sets
X_train, X_test, y_train, y_test = train_test_split(
    X, y,
    test_size=0.2,
    random_state=42,
    stratify=y
)
```

[5]:
```
# Step 6: Standardize the features
scaler = StandardScaler()

X_train = scaler.fit_transform(X_train)
X_test = scaler.transform(X_test)
```

[6]:
```
# Step 7: Create the KNN model
knn = KNeighborsClassifier(n_neighbors=5)

# Step 8: Train the model
knn.fit(X_train, y_train)

# Step 9: Predict the test data
y_pred = knn.predict(X_test)
```

[7]:
```
# Step 10: Calculate the confusion matrix
cm = confusion_matrix(y_test, y_pred)

print("\nConfusion Matrix:")
print(cm)
```

```
Confusion Matrix:
[[80 20]
 [26 28]]
```

[8]:
```
# Step 11: Calculate performance metrics
accuracy = accuracy_score(y_test, y_pred)

error_rate = 1 - accuracy

precision = precision_score(
    y_test, y_pred, zero_division=0
)

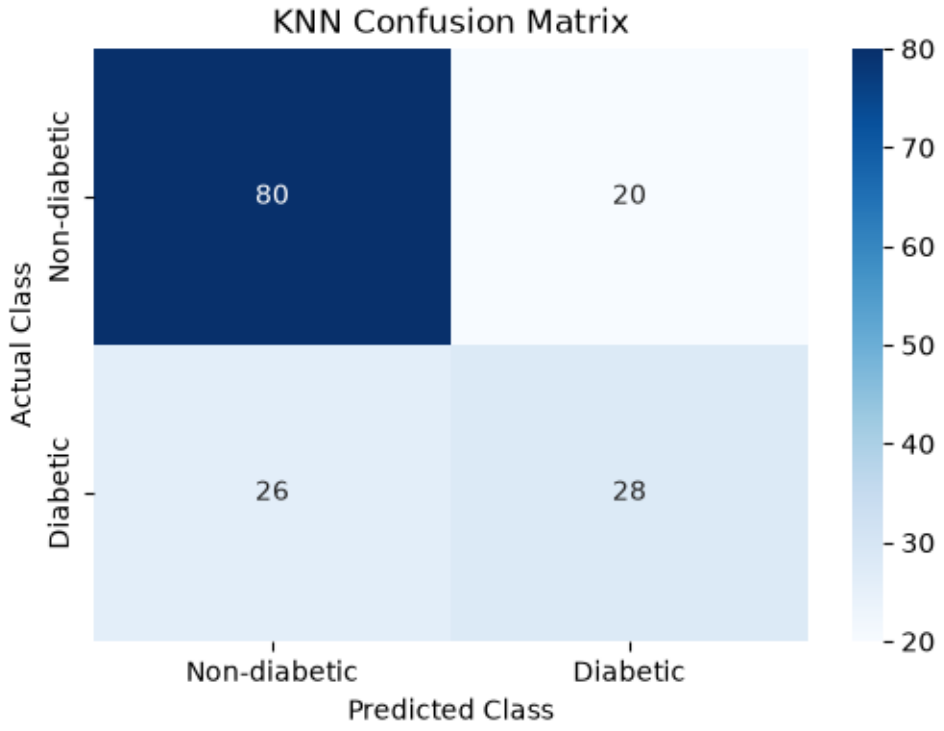
print("\nAccuracy:", round(accuracy * 100, 2), "%")
print("Error Rate:", round(error_rate * 100, 2), "%")
print("Precision:", round(precision * 100, 2), "%")
```

```
Accuracy: 70.13 %
Error Rate: 29.87 %
Precision: 58.33 %
```

[9]:
```python
# Step 12: Plot the confusion matrix
plt.figure(figsize=(6, 4))

sns.heatmap(
    cm,
    annot=True,
    fmt="d",
    cmap="Blues",
    xticklabels=["Non-diabetic", "Diabetic"],
    yticklabels=["Non-diabetic", "Diabetic"]
)

plt.title("KNN Confusion Matrix")
plt.xlabel("Predicted Class")
plt.ylabel("Actual Class")
plt.show()
```

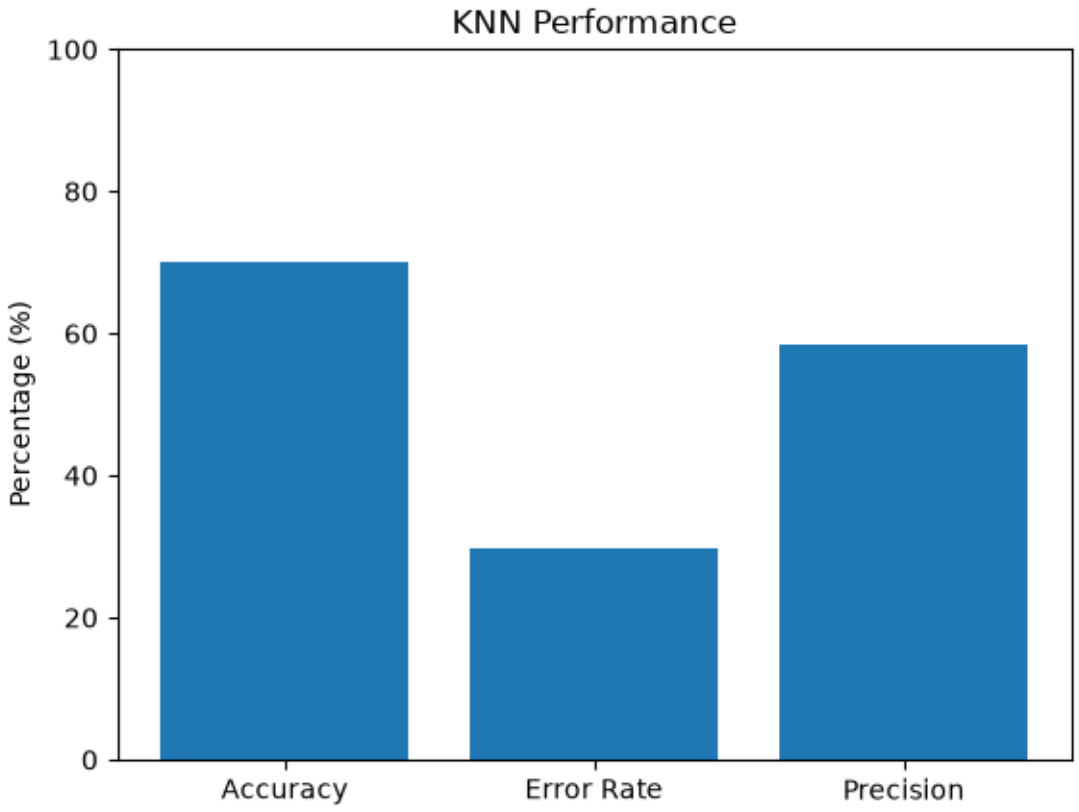


[10]:
```
# Plot performance metrics

plt.bar(
    ["Accuracy", "Error Rate", "Precision"],
    [accuracy * 100, error_rate * 100, precision * 100]
)

plt.title("KNN Performance")
plt.ylabel("Percentage (%)")
plt.ylim(0, 100)

plt.show()
```



[ ]: